Практична робота № 1

Тема: Дослідження мережевих протоколів засобами інструментів розробника

Мета роботи

- Опанувати інструменти дослідження мережевих протоколів HTTP та HTTPS (`curl`, графічний клієнт Postman, інструменти розробника веббраузера DevTools).
- Набути практичних навичок формування, передачі та структурного аналізу клієнт-серверних HTTP-запитів із різними методами (`GET`, `POST`, `PUT`, `DELETE`), статусами відповідей та керуючими заголовками.
- Дослідити механізми захищеного транспорту TLS/HTTPS, життєвий цикл і атрибути безпеки `Cookie` (`HttpOnly`, `Secure`, `SameSite`), а також функціонування політики безпеки CORS.

Технічне забезпечення

- Операційна система: Windows, macOS або Linux.
- Середовище розробки: Visual Studio Code (з розширенням REST Client або Thunder Client).
- Термінал / CLI: bash, zsh або PowerShell із встановленою утилітою `curl`.
- Інструменти тестування API: графічний клієнт Postman (або вебверсія/аналоги) та веббраузер із сучасними DevTools (Google Chrome, Firefox тощо).
- Система контейнеризації (опціонально для локального стенда): Docker Engine / Docker Desktop.
- Система контролю версій: Git.

Короткі теоретичні відомості

1. Анатомія протоколу HTTP та структура повідомлень

Протокол прикладного рівня HTTP (HyperText Transfer Protocol) побудований за текстовою архітектурою «клієнт-сервер». Кожна взаємодія складається з пари: **запит (Request)** та **відповідь (Response)**.

Запит клієнта містить:

- **Стартовий рядок (Request Line):** метод HTTP, відносний шлях ресурсу та версія протоколу (наприклад, `GET /api/v1/resource HTTP/1.1`).
- **Заголовки (Headers):** пари «ключ-значення», що несуть метадані (`Content-Type`, `Accept`, `Authorization`, `User-Agent`).
- **Порожній рядок-розділювач (`\r\n`).**
- **Тіло повідомлення (Body / Payload):** дані запиту (зазвичай у форматі JSON, Form Data або бінарному вигляді).

Відповідь сервера має симетричну структуру:

- **Статусний рядок (Status Line):** версія протоколу, цифровий тризначний код статусу та текстове пояснення (наприклад, `HTTP/1.1 200 OK`, `HTTP/1.1 201 Created`, `HTTP/1.1 404 Not Found`).
- **Заголовки відповіді:** метадані сервера (`Content-Type`, `Set-Cookie`, `Date`, `Cache-Control`).
- **Порожній рядок-розділювач.**
- **Тіло відповіді:** отриманий ресурс або повідомлення про помилку.

Семантика основних методів визначає їхню ідемпотентність та безпечність:

- `GET` — отримання представлення ресурсу (безпечний, ідемпотентний).
- `POST` — створення нового підлеглого ресурсу або передача даних на обробку (небезпечний, неідемпотентний).
- `PUT` — повне оновлення або створення ресурсу за фіксованим URI (небезпечний, ідемпотентний).
- `DELETE` — видалення ресурсу (небезпечний, ідемпотентний).
- `OPTIONS` — опитування можливостей сервера або виконання попереднього запиту (Preflight) у механізмі CORS.

---

2. Практична діагностика через утиліту `curl`

Консольна утиліта `curl` є інженерним стандартом для налагодження та тестування

мережевої взаємодії. Вона дозволяє надсилати довільні байти через різні протоколи та контролювати кожен параметр з'єднання.

Ключові параметри запуску `curl`:

- `-X, --request <METHOD>` — явно вказує метод HTTP (`GET`, `POST`, `PUT`, `DELETE`, `OPTIONS`).
- `-H, --header <"Key: Value">` — додає довільний заголовок до запиту (може повторюватися декілька разів).
- `-d, --data <DATA>` — передає тіло запиту (за замовчуванням автоматично вмикає метод `POST`).
- `-i, --include` — виводить HTTP-заголовки відповіді сервера разом із тілом.
- `-I, --head` — надсилає метод `HEAD` і виводить виключно HTTP-заголовки без тіла.
- `-v, --verbose` — вмикає детальний діагностичний режим (показує резолвінг IP-адреси, процес TCP handshake, узгодження TLS та сирі HTTP-заголовки).
- `-s, --silent` — приховує шкалу прогресу та зайві технічні повідомлення.
- `-c <file>` — записує отримані від сервера cookies у текстовий файл (cookie jar).
- `-b <file|string>` — передає cookies на сервер із файлу або у вигляді рядка.
- `-w, --write-out <format>` — форматує виведення службової інформації та метрик часу.

Приклад дослідження публічного API GitHub (не потребує авторизації для базових запитів):

```
# 1. Запит заголовків ресурсу з виведенням статусу
curl -I https://api.github.com/zen ①

# 2. Отримання даних користувача у форматі JSON з кастомним Accept заголовком
curl -s -H "Accept: application/vnd.github.v3+json"
 https://api.github.com/users/octocat ②

# 3. Вимірювання часових затримок з'єднання (Time to First Byte та загальний час)
curl -s -o /dev/null -w "DNS: %{time_namelookup}s | Connect: %{time_connect}s |
 TTFB: %{time_starttransfer}s | Total: %{time_total}s\n" https://api.github.com/zen
 ③
```

① Отримання виключно метаданих відповіді без завантаження тіла.

② Додавання заголовка узгодження контенту (Content Negotiation) для отримання специфічного формату API.

③ Форматований профіль таймінгів мережевого стеку для оцінки затримок.

---

3. Захищене з'єднання HTTPS, рукостискання TLS та сертифікати

Протокол HTTPS (HTTP Secure) являє собою надбудову над класичним HTTP, де транспортний

потік шифрується криптографічним протоколом TLS (Transport Layer Security). Робота здійснюється через стандартний мережевий порт 443 (замість незахищеного порту 80).

Процес встановлення захищеного з'єднання (TLS Handshake) включає:

- **ClientHello:** клієнт надсилає підтримувані версії TLS (TLS 1.2, TLS 1.3), перелік підтримуваних шифронаборів (Cipher Suites) та розширення SNI (Server Name Indication).
- **ServerHello:** сервер обирає оптимальну версію TLS та узгоджений алгоритм шифрування.
- **Certificate & Key Exchange:** сервер передає цифровий ланцюжок сертифікатів X.509, підписаний довіреним центром сертифікації (CA), та бере участь у генерації спільних сесійних ключів шифрування.
- **Finished:** перехід на симетричне потокове шифрування (наприклад, AES-GCM або ChaCha20) для подальшого обміну HTTP-пакетами.

Утиліта `curl` у режимі вербозного виводу (`-v`) позначає етапи префіксами:

- `*` — технічна інформація та етапи узгодження TLS / TCP;
- `>` — байти, надіслані клієнтом на сервер (вихідний HTTP-запит);
- `<` — байти, повернуті сервером клієнту (вхідна HTTP-відповідь).

---

4. Збереження стану (Cookies) та політика безпеки CORS

Оскільки HTTP є протоколом без збереження стану (stateless), сервер використовує механізм `Cookie` для відстеження сесій клієнта:

- Сервер повертає клієнту заголовок `Set-Cookie: name=value; Path=/; HttpOnly; Secure; SameSite=Strict`.
- Клієнт зберігає значення і автоматично додає заголовок `Cookie: name=value` до всіх наступних запитів у межах вказаного домену та шляху.

Атрибути безпеки cookies:

- `HttpOnly` — блокує доступ до cookie з JavaScript-коду браузера (`document.cookie`), захищаючи від атак крадіжки сесії через XSS.
- `Secure` — дозволяє передачу cookie виключно через зашифроване з'єднання HTTPS.
- `SameSite` — регулює поведінку надсилання cookies при міжсайтових переходах (`Strict`, `Lax`, `None`), що слугує захистом від CSRF-атак.

CORS (Cross-Origin Resource Sharing) — це механізм безпеки браузера, що запобігає несанкціонованому читанню відповідей одного домену скриптами іншого (Same-Origin Policy). Для методів зі зміною стану (`POST`, `PUT`, `DELETE` або при наявності нестандартних заголовків) браузер спочатку надсилає запит попередньої перевірки **Preflight (`OPTIONS`)** із

заголовками:

- `Origin: https://myclient.com`
- `Access-Control-Request-Method: POST`
- `Access-Control-Request-Headers: Content-Type, Authorization`

Сервер повинен відповісти заголовками дозволу:

- `Access-Control-Allow-Origin: https://myclient.com` (або `*`)
- `Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS`
- `Access-Control-Allow-Headers: Content-Type, Authorization`

Постановка задачі

Для успішного проєктування та налагодження сучасних бекенд-систем на NestJS інженер повинен володіти низькорівневим розумінням протоколів прикладного та транспортного рівнів.

У межах практичної роботи необхідно провести дослідження мережевої взаємодії з REST API, протестувати CRUD-операції через графічний клієнт Postman, виконати діагностику запитів через термінальну утиліту `curl`, розібрати внутрішню структуру рукостискання TLS, дослідити поведінку сесійних cookies та перевірити заголовки політики CORS.

Для тестування використовуються відкриті платформи без необхідності генерації API-ключів:

- **`https://dummyjson.com/`** — повноцінне REST API для виконання операцій з ресурсами (товари, автентифікація, користувачі).
- **`https://httpbin.org/`** (або дзеркало `https://httpbingo.org/`) — сервіс-ехо для дослідження HTTP-заголовків, кодів відповідей, cookies та перевірки статусів.
- **(Опціонально для роботи офлайн)**: локальний Docker-контейнер `httpbin`:

  ```
  docker run -d -p 8080:80 kennethreitz/httpbin
  ```

  (У разі використання локального стенда адреси мають вигляд `http://localhost:8080/...`).

Завдання

Практична робота складається з трьох послідовних рівнів. Кожен наступний рівень базується на попередньому та розширює функціональність розроблюваного рішення.

Рівень 1. Базовий: дослідження HTTP через DevTools та Postman

1. **Підготовка середовища:**
   - Перевірити наявність та доступність інструментів `curl` у терміналі (`curl --version`).
   - Відкрити графічний клієнт Postman або налаштувати розширення REST Client у VS Code.
2. **Дослідження методів REST API в Postman:**
   - Виконати запит `GET https://dummyjson.com/products/1` — проаналізувати отримане тіло JSON, код статусу `200 OK`, час відповіді та розмір завантажених даних.
   - Виконати запит `POST https://dummyjson.com/products/add` — передати у тілі (Body → raw JSON) об'єкт нового товару (поля `title`, `price`). Проаналізувати код статусу `201 Created` та отриманий згенерований `id`.
   - Виконати запит `PUT https://dummyjson.com/products/1` — надіслати змінене поле `title` та перевірити повернений результат.
   - Виконати запит `DELETE https://dummyjson.com/products/1` — зафіксувати код статусу та наявність позначки видалення `isDeleted: true`.
   - Зберегти створені запити у колекцію Postman (або у файл `requests.http`).
3. **Аналіз структури в DevTools:**
   - Відкрити у веббраузері адресу `https://dummyjson.com/products` із відкритою панеллю **DevTools (вкладка Network)**.
   - Знайти відповідний запит у списку мережевої активності та зафіксувати в окремій таблиці:
     - General: Request URL, Request Method, Status Code, Remote Address.
     - Response Headers: `content-type`, `date`, `server`, `etag`.
     - Request Headers: `accept`, `user-agent`, `accept-encoding`.
     - Вкладку Timing: проаналізувати тривалість фаз DNS Lookup, Initial connection, Waiting for server response (TTFB), Content Download.

Рівень 2: консольна діагностика мережевої взаємодії через `curl`

1. **Базові виклики та робота з методами:**
   - Надіслати запит `GET` на отримання списку користувачів `https://dummyjson.com/users?limit=2&select=firstName,email` через термінал за допомогою `curl`.
   - Виконати запит додавання сутності методом `POST` із передачею JSON-рядка через прапорець `-d` та явним встановленням заголовка `Content-Type: application/json` за допомогою прапорця `-H`:

- Ресурс: `https://dummyjson.com/posts/add`
- Тіло: об’єкт із полями `title` та `userId`.

2. **Аналіз заголовків та статусів без завантаження тіла:**
   - Виконати команду `curl -I` для ресурсу `https://dummyjson.com/products`. Зафіксувати отримані заголовки сервера.
   - Здійснити запит до `https://httpbin.org/status/404` та `https://httpbin.org/status/500` з прапорцем `-i`, щоб переконатися у виведенні заголовків із відповідними статусами помилок.
3. **Автентифікація через заголовок та передача токена:**
   - Отримати тестовий токен автентифікації, надіславши POST-запит до `https://dummyjson.com/auth/login` з обліковими даними:
     - `username: "emilys"`
     - `password: "emilyspass"`
   - Витягти отриманий `accessToken` та виконати наступний запит до захищеного ендпоінта `https://dummyjson.com/auth/me`, передавши токен у заголовку `Authorization: Bearer <TOKEN>` через прапорець `-H`. Зафіксувати успішну відповідь із даними користувача.
4. **Вимірювання мережевих метрик (Performance profiling):**
   - Скласти запит із використанням прапорця `-w` для отримання детальної розбивки таймінгів з’єднання з `https://dummyjson.com/products`:
     - Час DNS резолвінгу (`%{time_namelookup}s`);
     - Час TCP з’єднання (`%{time_connect}s`);
     - Час узгодження TLS (`%{time_appconnect}s`);
     - Час очікування першого байта сервера TTFB (`%{time_starttransfer}s`);
     - Загальний час операції (`%{time_total}s`).

Рівень 3. Просунутий: глибинний аналіз HTTPS/TLS, Cookie та CORS

1. **Аналіз TLS Handshake через детальне трасування:**
   - Виконати запит `curl -v https://dummyjson.com/products/1`.
   - Зафіксувати у звіті та розібрати ключові етапи узгодження:
     - Встановлення TCP-з’єднання з IP-адресою та портом 443;
     - Відправлення `ClientHello` та отримання `ServerHello`;
     - Перевірка сертифіката (видавець CA, термін дії, Subject Alternative Name);
     - Узгоджений шифронабір (Cipher Suite) та протокол (наприклад, TLSv1.3 або TLSv1.2).

- Порівняти поведінку при примусовому зверненні через незахищений протокол HTTP:
  - Виконати `curl -I http://dummyjson.com/products/1`.
  - Зафіксувати повернений статус перенаправлення `301 Moved Permanently` або `308 Permanent Redirect` та наявність заголовка `Location: https://...`.

2. **Дослідження життєвого циклу `Cookie` через Cookie Jar:**
   - Надіслати запит до `https://httpbin.org/cookies/set?user_role=student&session_key=lab1_token` із збереженням отриманих cookies у файл за допомогою прапорця `-c cookies.txt`.
   - Дослідити вміст згенерованого текстового файлу `cookies.txt` (формат Netscape cookie: домен, прапорець захищеності, шлях, термін життя, ім'я та значення).
   - Здійснити повторний запит до ендпоінта перевірки `https://httpbin.org/cookies`, передавши збережені cookies за допомогою прапорця `-b cookies.txt`. Переконатися, що сервер розпізнав надіслані cookies у тілі відповіді.
3. **Аналіз заголовків безпеки Cookie:**
   - Надіслати запит до `https://httpbin.org/response-headers` із передачею кастомного заголовка `Set-Cookie` через query-параметри:
     - Параметр: `Set-Cookie=session_id=xyz789;%20Path=/;%20Secure;%20HttpOnly;%20SameSite=Strict`
   - Перевірити наявність та значення атрибутів `HttpOnly`, `Secure` і `SameSite` у відповіді через `curl -i`.
   - Пояснити у звіті, від яких саме векторів атак захищає кожен із цих атрибутів.
4. **Симуляція перевірки політики CORS (Preflight Request):**
   - Використовуючи `curl`, симулювати попередній запит браузера методом `OPTIONS` до ресурсу `https://httpbin.org/post` (або `https://dummyjson.com/products/add`), передавши заголовки:
     - `-H "Origin: https://my-college-app.edu"`
     - `-H "Access-Control-Request-Method: POST"`
     - `-H "Access-Control-Request-Headers: Content-Type, Authorization"`
   - Проаналізувати заголовки відповіді сервера: чи присутні `Access-Control-Allow-Origin`, `Access-Control-Allow-Methods`, `Access-Control-Allow-Headers` та який статус повернув сервер (`200 OK` або `204 No Content`).

Приклад очікуваного виведення програми

```
=== ЗАПИТ ДО АВТОРИЗОВАНОГО РЕСУРСУ ЧЕРЕЗ CURL ===
```

```
> POST /auth/login HTTP/1.1
> Host: dummyjson.com
> Content-Type: application/json
> Accept: application/json

< HTTP/1.1 200 OK
< Content-Type: application/json; charset=utf-8
< Connection: keep-alive
{
  "id": 1,
  "username": "emilys",
  "email": "emily.johnson@x.dummyjson.com",
  "firstName": "Emily",
  "lastName": "Johnson",
  "gender": "female",
  "image": "https://dummyjson.com/icon/emilys/128",
  "accessToken": "eyJhbGciOiJIUzI1NiIsIn..."
}

=== ПЕРЕВІРКА МЕТРИК ПРОДУКТИВНОСТІ МЕРЕЖЕВОГО З'ЄДНАННЯ ===
DNS Lookup: 0.014210s
TCP Connect: 0.048512s
TLS Handshake: 0.092140s
Time To First Byte (TTFB): 0.185210s
Total Transaction Time: 0.215430s

=== ДОСЛІДЖЕННЯ COOKIE JAR У ТЕРМІНАЛІ ===
# Netscape HTTP Cookie File
httpbin.org FALSE   /   FALSE   0   session_key lab1_token
httpbin.org FALSE   /   FALSE   0   user_role   student

< GET /cookies HTTP/1.1
< Host: httpbin.org
{
  "cookies": {
    "session_key": "lab1_token",
    "user_role": "student"
  }
}
```

Контрольні питання

1. Чим відрізняється стартовий рядок HTTP-запиту від статусного рядка HTTP-відповіді? Які обов’язкові компоненти вони містять?
2. Яка різниця між безпечними (safe) та ідемпотентними (idempotent) методами HTTP? До яких категорій належать методи `GET`, `POST`, `PUT` та `DELETE`?
3. Що означають коди стану відповідей із груп `2xx`, `3xx`, `4xx` та `5xx`? Наведіть конкретні

приклади кодів `201`, `301`, `401`, `403`, `404` та `500`.

4. Для чого призначені керуючі заголовки `Content-Type` та `Accept`? Як клієнт і сервер домовляються про формат даних (Content Negotiation)?
5. Опишіть етапи рукостискання TLS Handshake. Чому передача конфіденційних даних за протоколом HTTP через відкриту мережу є небезпечною?
6. Що таке Time To First Byte (TTFB) та з яких часових затримок він складається під час виконання запиту?
7. Яке призначення атрибутів `HttpOnly`, `Secure` та `SameSite` у заголовку `Set-Cookie`? Від яких типів вразливостей вони захищають користувача?
8. Що таке Preflight-запит у механізмі CORS? У яких випадках веббраузер зобов'язаний надіслати запит методом `OPTIONS` перед основним запитом?

Критерії оцінювання

Оцінювання здійснюється за шкалою повноти реалізації рівнів практичної роботи:

- **«Задовільно» (Рівень 1):** реалізовано базовий прототип, створено структуру проєкту, програма успішно компілюється та виконує базові сценарії.
- **«Добре» (Рівень 2):** виконано базовий рівень та реалізовано ключову функціональність заняття, забезпечено надійність і безпеку виконання коду.
- **«Відмінно» (Рівень 3):** повністю завершено всі три рівні завдання з дотриманням кращих інженерних практик, чистоти архітектури та оптимізації.
- **Усний захист:** обов'язкове пояснення логіки коду та впевнені відповіді на контрольні питання викладача.

Вимоги до звітності

1. Опублікувати вихідний код у Git-репозиторії.
2. Оформити файл `README.md` з описом проєкту, інструкцією запуску та обов'язковими скриншотами роботи програми (включаючи демонстрацію відсутності помилок).
3. Продемонструвати роботу програми викладачеві та захистити результати.